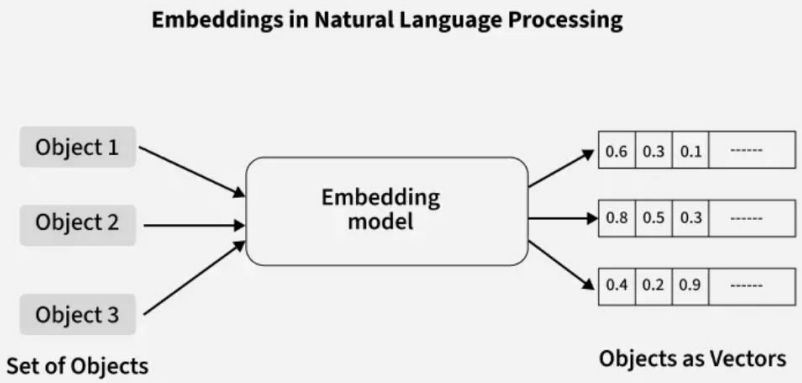
Natural Language processing

Section 3

Word Embeddings

“numerical representations of words as dense vectors in a lower-dimensional space, designed so that words with similar meanings are close to each other in that space.”



Word Embedding

Cat →	1.2	-0.1	4.3	3.2
Mat →	0.4	2.5	-0.9	0.5
on →	2.1	0.3	0.1	0.4

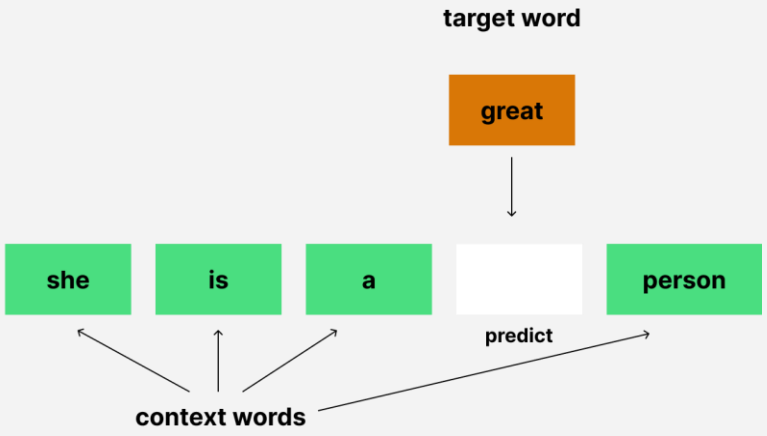
‘Word Embedding using Word2Vec

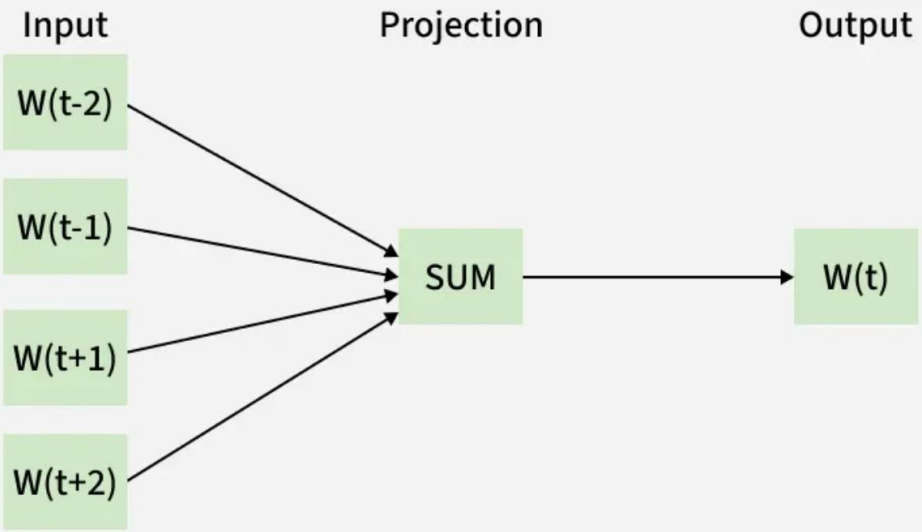
Word2Vec

- **Word2Vec** is a word embedding technique in natural language processing (NLP) that allows words to be represented as vectors in a continuous vector space.
- Researchers at Google developed **word2Vec** that maps words to high-dimensional vectors to capture the semantic relationships between words.
- It works on the principle that words with similar meanings should have similar vector representations. Word2Vec utilizes two architectures..

1- CBOW : Continuous Bag of Words

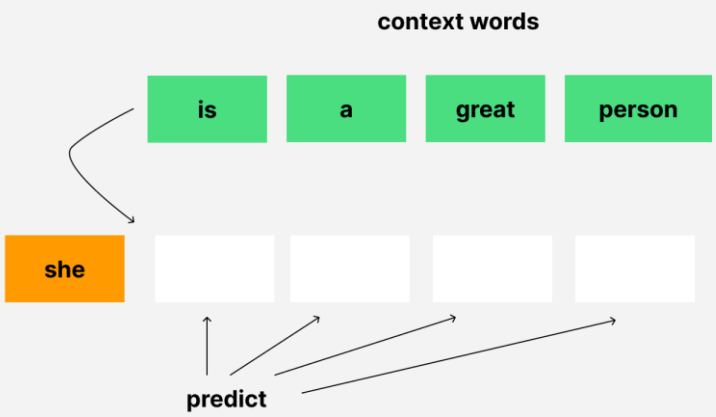
predicts the current word given context words within a specific window.

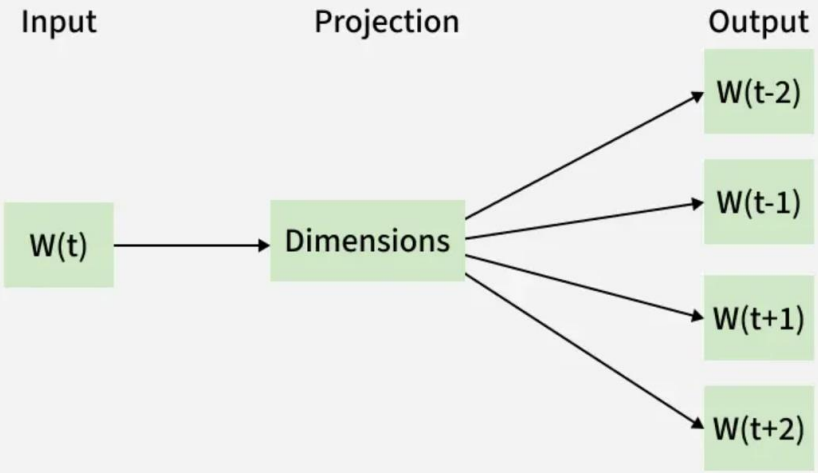




2- skip gram

predicts the surrounding context words within specific window given current word.





1. Importing Required Libraries

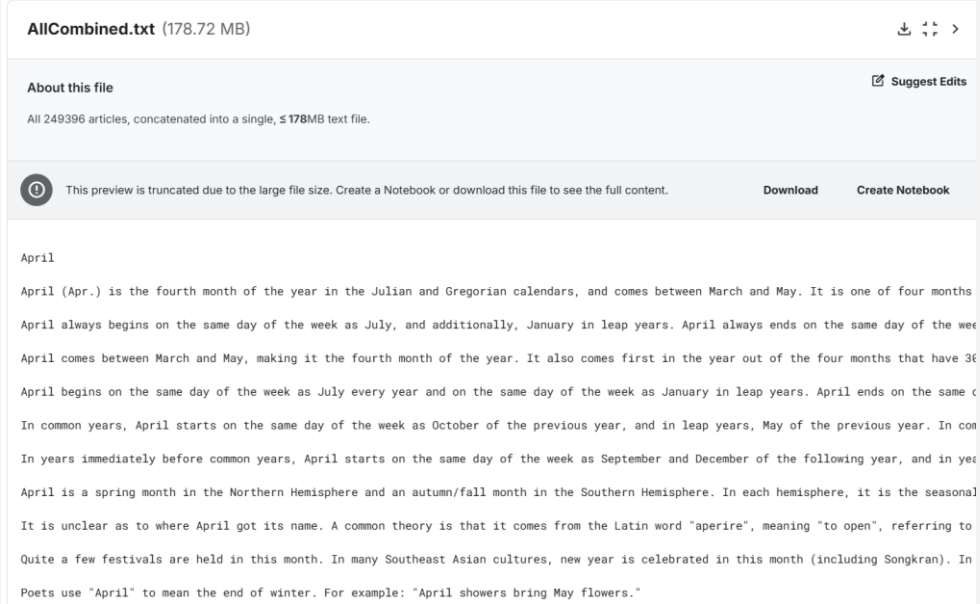
We will import necessary **NLTK** and **Gensim** for building the Word2Vec model and processing text.

Gensim is an open-source Python library for Natural Language Processing (NLP), with a primary focus on unsupervised topic modeling and semantic vector representation of text. Its name, "Generate Similar," reflects its core functionality in identifying semantic similarities within textual data.

```
pip install gensim
```

2. Choose and load the dataset

We will use the **Plain text Wikipedia (SimpleEnglish) from Kaggle** which is an Unsupervised text corpus of all 249,396 articles in the Simple English Wikipedia.



Loading and preprocessing the dataset

```python
# Import libraries
from gensim.models import Word2Vec
import gensim
from nltk.tokenize import sent_tokenize, word_tokenize

file_path = "english_wiki.txt"

# read and clean the data file
with open(file_path, 'r', encoding='utf-8', errors='ignore') as file:
    content = file.read()
    cleaned_text = content.replace("\n", " ")
    print("File loaded successfully!")

# prepare data for training
data = []

# Split the text into sentences
for i in sent_tokenize(cleaned_text):
    temp = []

    # Tokenize each sentence into words
    for j in word_tokenize(i):
        temp.append(j.lower())

    data.append(temp)

print("Data prepared successfully!")
```

Using the CBOW architecture: sg=0

```python
model_cbow = Word2Vec(
    sentences=data,      #training data
    vector_size=50,      #word vector size
    window = 5,          #context window
    min_count=1,         #do not ignore rare words
    sg =0                #CBOW architecture
)

print('CBOW Model trained successfully')
```

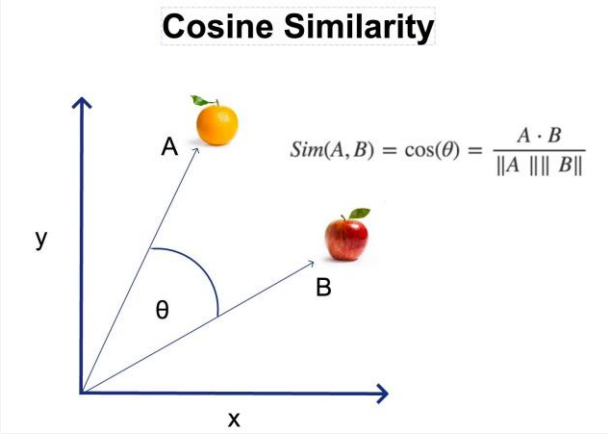
Using the skip gram architecture : sg=1

```python
model_sg = Word2Vec(
    sentences=data,      #training data
    vector_size=50,      #word vector size
    window = 5,          #context window
    min_count=1,         #do not ignore rare words
    sg =1                #skip gram architecture
)

print('Skip gram Model trained successfully')
```

Cosine similarity between 2 words for the 2 models

This measures how similar two word vectors are.




```python
#Cosine similarity between 2 words in the CBOW
print("Cosine similarity between 'April' and 'Spring' - CBOW",
      model_cbow.wv.similarity('april','spring'))

#Cosine similarity between 2 words in the skip gram
print("Cosine similarity between 'April' and 'Spring' - Skip gram",
      model_sg.wv.similarity('april','spring'))
```

```
Cosine similarity between 'April' and 'Spring' - CBOW 0.63737035
Cosine similarity between 'April' and 'Spring' - Skip gram 0.9743924
```

Find Top-n most similar to a word

```python
#Top-n most similar - CBOW
print("\nTop 5 words most similar to 'april': ")
print(model_cbow.wv.most_similar('april',topn=5))

#Top-n most similar - Skip gram
print("\nTop 5 words most similar to 'april': ")
print(model_cbow.wv.most_similar('april',topn=5))
```

```
Top 5 words most similar to 'april' CBOW Model:
[('are', 0.990347683429718), ('in', 0.9899370670318604), ('had', 0.9898755550384521), ('the', 0.9898548126220703), ('arithmetic', 0.9897503852844238)]

Top 5 words most similar to 'april' Skip-gram Model:
[('are', 0.990347683429718), ('in', 0.9899370670318604), ('had', 0.9898755550384521), ('the', 0.9898548126220703), ('arithmetic', 0.9897503852844238)]
```

See you soon